A ROBONUGGETS GUIDE

The new ARMS levels

Agentic OS v2: twelve new levels across Skills, Routines, Memory and Apps, with a copy-paste Claude Code prompt that sets up each one.

README

What this is. The guide that goes with the Agentic OS v2 video. It uses the ARMS framework (Apps, Routines, Memory, Skills), with three up-to-date levels for each part, so twelve in all. Every prompt shown in the video is here word for word, and each level has at least one prompt you can paste into Claude Code. All text is selectable.

How to use it. The fastest way: send this whole PDF to your AI agent and ask which three levels would help your setup most. It can compare each level with your skills, scheduled tasks, CLAUDE.md, memory files and connected apps, then pick out the prompts you need. To read it yourself, go part by part in the order the video teaches: Skills, Routines, Memory, then Apps. Each part has a page on what the levels are, then a page of prompts. Page 3 maps all twelve levels, and the checklist on page 16 lets you tick them off.

Want my exact setup? The RUBRIC Agentic OS template and the micro apps from the video live in the RoboNuggets community classroom. This guide gives you the ideas and the prompts to build your own.

What's new

Three things changed how I run Claude in the past weeks, and they show up all through this guide:

Claude 5.5	The newest Claude models, a big step up from the last ones. They're smart enough to find their way around a big workspace with plain text alone, and they have a 1 million token context window.
Jev	A fast, cheap decision model from TypeSafe, reached through OpenRouter (one account that reaches many AI models). It sorts things rather than writing them, so it can decide which model should take each prompt in well under a second.
Mods	Small add-ons that change how the Claude Code desktop app looks and works. Think of them as Chrome extensions for Claude Code.

The twelve ARMS levels

ARMS is like giving your agent arms to reach into your workspace. If you set these four parts up the right way, you'll be ahead of 99% of people using AI agents. The video goes bottom up, so skip ahead to the first level you haven't built yet.

	LEVEL 1	LEVEL 2	LEVEL 3
S **Skills**	**Know your skill types** Database, artifact and self-triggered skills, plus wildcards so references don't box you in. Page 4	**Model routing** Each skill runs on the smallest model that does the job well. Page 4	**Self-improving skills** Every piece of feedback makes the skill better next time. Page 4
R **Routines**	**Active vs passive** Most routines should only ping you when something is worth flagging. Page 7	**Capped routines** Know what each routine costs, and cap the expensive ones. Page 7	**Routines that suggest routines** Claude spots the jobs you keep doing by hand. Page 7
M **Memory**	**Long-term memory** Plain-text maps so Claude finds anything fast and cheap. Page 9	**Short-term memory** Compact with a purpose and keep your cache warm. Page 10	**Self-evolving memory** Never correct Claude on the same thing twice. Page 10
A **Apps**	**Connect anything** Even apps with no official connector, lightest way in first. Page 12	**Mods** Change how Claude Code works, then mod your browser too. Page 12	**A micro app for every big job** Move routine work out of the chat window. Page 12

THE IDEA BEHIND ALL TWELVE

The visuals of an Agentic OS are only part of the value. What makes it work is how you organise your context and workspace underneath. Each level here makes your setup cheaper, faster or better, and it keeps working as new models come out.

Skills

Skills are saved instructions Claude follows whenever you call them.

L1 Know your skill types

The types depend on your work. Here are the three I use most, plus one trap to watch for:

- **Database skills** hold knowledge, not just steps. My **/tone** skill holds the examples and principles of how I write, so any chat can draft in my voice. My **/thumb** skill points to every thumbnail we've shipped. If you make or reference the same kind of output every week, give it a database skill.
- **Watch for overfitting.** References can lock your agent into copying them, so you stop getting fresh ideas. Have database skills add wildcards by default: if Claude pitches 10 titles, 8 follow your references and 2 are wildcards. Add the good wildcards to your references. My **/wildcard** skill defines what a wildcard means.
- **Artifact skills** make decks, infographics and PDFs. Agents are great at HTML (the language web pages are built in), so build in HTML first and export at the end. Tie the skill to a brand book in HTML (your colours and fonts) to avoid the generic AI look. Then give your artifacts one inbox, so you find old ones without spending tokens.
- **Self-triggered skills** run outside the chat, from a button. I click **/clean-up** when my computer slows down. It runs on **claude -p**, which sends Claude a prompt in the background without opening a chat. Pick the model and effort next to the button.

L2 Model routing

A common mistake is running every skill on a big, pricey model. Skills with clear-cut steps can run on Sonnet or even Haiku. My **/grammar** skill only checks spelling, so it has **model: haiku** at the top of its file. For everything else, Jev picks the model for each prompt. In the past weeks it made 4,451 routing calls, each about 0.6 seconds, for 18 cents in total, and about 4 in 10 went to Haiku. I never felt the smaller model.

- A skills view that ranks your skills by tokens shows which ones to slim down. Mine is part of RUBRIC Agentic OS.

L3 Self-improving skills

The more you use a skill, the better it should get. When I review a video, I pin comments on the exact moment, copy them all and send them to Claude. A teach switch adds one line asking Claude to update the **/cut** and **/animate** skills with what it learned. It's a simple version of reinforcement learning, the same idea AI labs use to improve their models.

- The review app with the teach switch is RUBRIC Editor, in the RoboNuggets classroom.

LEVEL 1 - DATABASE SKILL

PASTE INTO CLAUDE CODE

make me a database skill for something i reference every week.
- ask me which one first: my tone of voice, my best titles, my thumbnails or my links
- keep the examples and principles in a reference file, and the skill.md short
- when it pitches ideas, 8 of 10 follow my references and 2 are wildcards
- when i like a wildcard, offer to add it to the references
show me the skill before you save it.

LEVEL 1 - ARTIFACT SKILL

PASTE INTO CLAUDE CODE

set up one artifact skill, so everything you make looks like my brand.
- build a brand book as one html page: my colours, fonts, spacing and logo rules
- make a skill that builds every deck, infographic and pdf in html first, from that brand book
- export to pdf or images only at the end
- save every artifact to one folder with an index page, newest first, that i can search without asking you
show me one sample page before you save the skill.

LEVEL 1 - SELF-TRIGGERED SKILL

AS SHOWN IN THE VIDEO

Set up a self-triggered skill for me.
- Pick a skill I run often, like a computer clean-up.
- Add a run button for it to my dashboard.
- On click, run it in the background with claude -p.
- Let me pick the model and effort next to the button.
- Save a short report each time it finishes.

No dashboard yet? Ask Claude for a plain local page with one button. **claude -p** is Claude Code's headless mode: it runs one prompt and exits, with no chat window.

LEVEL 2 - MODEL ROUTING

AS SHOWN IN THE VIDEO

run each of my skills on the right size of model.
- pin simple fixed-step skills to the smallest model
- for the rest, let a fast router pick: Jev from TypeSafe, via OpenRouter
- read OPENROUTER_API_KEY from my computer, never ask in chat
- log each pick, its time and cost, with an on/off switch
if something isn't possible here, offer the closest fix.

You need an OpenRouter account and API key for the Jev part. Keep the key in a file on your computer, never in the chat. Pinning small skills works without it.

LEVEL 3 - SELF-IMPROVING SKILL

PASTE INTO CLAUDE CODE

make my most-used skill get better every time i give it feedback.
- find which skill i run most, from my past sessions
- when i give feedback on its output, find the principle behind it
- fold that principle into the skill, so it gets sharper, not longer
- show me the change before you save it
- keep a short log of what changed and why
start with one skill. we'll add more later.

The best fix is a principle that explains the miss, not another rule stacked on the last one. A good update often makes the skill shorter.

Routines

Routines are tasks Claude runs on a schedule, without you typing anything.

L1 Active vs passive routines

Active routines hand you an output to review, like my daily inbox digest that lands in Telegram. **Passive routines** run in the background and only ping you when they find something worth flagging. My weekly brand watch checks the web for sites using the RoboNuggets brand without our approval. If it finds none, I hear nothing.

This matters as you grow. If all 50 of your routines were active, you'd have 50 things to review every week. Good passive ideas:

- A domain scout that pings you when a good name for your business becomes available.
- Machine care that clears old files and flags apps slowing your computer down.

L2 Capped routines

Every scheduled task costs you something, either plan usage or API credits. Once you have a lot of them, it's easy to lose track of the expensive ones. I keep a routines view of the week's tasks, sorted by the tokens each one uses, so I can spot any that drain usage without adding value.

When one costs too much, ask Claude to optimise it and cap it. A routine burning about 18 million tokens, or $10 a week in API credits, can often come down to a $4 cap. Claude usually finds a lot of fat to cut.

- My routines view is part of RUBRIC Agentic OS, in the RoboNuggets classroom.

L3 Routines that suggest routines

The best routines are jobs you already do by hand, but you often don't notice you repeat them. Every Friday morning, a scheduled task reads my week of sessions, spots the jobs I keep doing myself and suggests routines. It puts them in RUBRIC Recos, a small app that runs on my computer. Before my usage resets, I add the good ones to a cart, copy it and send it to Claude to set up. Every week the system gets a little better.

- RUBRIC Recos is in the RoboNuggets classroom.

LEVEL 1 - PASSIVE ROUTINES

PASTE INTO CLAUDE CODE

go through my scheduled tasks and sort them into active and passive.
- active: hands me an output to review every run
- passive: only messages me when it finds something worth flagging
- turn any that don't need my review every time into passive ones
- suggest 3 new passive routines for my business, like a brand watch, a domain scout or machine care
change nothing until i pick.

LEVEL 2 - CAPPED ROUTINES

PASTE INTO CLAUDE CODE

find which of my scheduled tasks costs the most.
- check each one's recent runs: tokens per week, and what that would cost on the api
- rank them, most expensive first
- for the top one, cut the fat until it fits a cap i set, like $4 a week
- show me the cost before and after, and what it still does
don't drop anything i rely on without asking.

LEVEL 3 - A ROUTINE THAT SUGGESTS ROUTINES

AS SHOWN IN THE VIDEO

every friday, suggest routines for jobs i keep doing by hand.
- check my past week of sessions for jobs done 3+ times
- one short card each: what it does, when, what it saves
- show the cards on a page with a cart i copy back to you
- set up only the ones i pick, before my usage resets
change nothing until i pick.

Memory

Memory is everything Claude knows about you and your work, and just as important, how fast and how cheaply it can find it.

L1

Long-term memory

When people say "second brain" these days, they mean the files in your workspace, which is probably just a folder on your computer. Mine holds about 420,000 files. A fancy map of them is optional. What matters is speed and cost: can Claude find your stuff fast, without burning through your usage limits?

My setup is simpler than most: three plain-text layers that give Claude a map of everything.

1. **MEMORY.md** - Claude Code loads it at the start of every chat. Keep one short line per memory, like a library's card catalog, so Claude checks the catalog before it walks the shelves.
2. **CLAUDE.md as a map** - Claude also reads this file at the start of every chat. Instead of a rulebook, I use mine as a map that sends Claude to each part of my work. Each department then has its own router file, like **content.md**, that points to its folders. If you also use Codex, keep shared rules in one **AGENTS.md** and point **CLAUDE.md** to it.
3. **Database skills as routers** - a skill like **/yourbusiness-data** that tells Claude where the up-to-date numbers and dashboards live.

Plain text is enough. I don't use a vector database. Boris Cherny, who created Claude Code, has said early versions used one, and the team found that letting Claude search the files itself worked better. If it works for Anthropic, it will likely work for your business too.

Give images a text description. Claude reads text, so searching 1,000 untagged images means looking at every one. I save the prompt that made each image as its description. For older images, a cheap model like Gemini 3.5 Flash-Lite can write one. It tagged about 2,400 of mine for about $2.40.

A QUICK TEST

Ask Claude where something lives in your workspace, like last month's numbers or your latest client deck. If it opens two or three files to answer, your map works. If it searches everywhere, your map needs another layer.

L2 Short-term memory

This is the context window, Claude's working memory for one chat. The Claude 5.5 models hold 1 million tokens. It comes with two problems:

- The fuller a chat gets, the worse Claude gets at recalling what's in it. Anthropic calls this context rot.
- Every message rereads the whole chat, so a bloated chat costs more and drains your usage faster.

Fix 1: compact with a purpose. When a chat fills up, Claude compacts it, which means it writes itself a summary. Tell it what to keep, like **/compact keep the business decisions and the templates**, or Claude decides and might drop the one detail you need. You can also give Claude standing rules for what to always keep.

Fix 2: watch your cache. The prompt cache lets Claude reread a chat for a small fraction of the normal price. But leave a chat idle for about an hour and the cache goes cold, so your next message pays to load the whole chat again. One chat of mine had been idle for 11 and a half hours. My next message would have reloaded about 600,000 tokens, about $5 at API prices. My usage mod shows the cache state next to my 7-day and 5-hour limits. My chats also send themselves a tiny ping about once an hour to stay warm, for up to 4 hours. Over the last 30 days, that would have saved me about $437 at API prices.

- The usage mod is free: github.com/robonuggets/ribbon

L3 Self-evolving memory

If you have to correct Claude on the same thing twice, your memory system missed something. I use three things:

- **A fix backlog.** When I notice mid-job that a skill or app should do better, I type **/note** and my feedback goes into that skill's own file. Fixing it right then would lose my thread. When I'm ready, or my usage is about to reset, Claude applies the whole list.
- **/calibrate** for feedback that really matters. Claude Code saves some memories on its own, but you don't want to leave the important ones to chance. It saves what I taught it, links it to related memories and fixes the skill it belongs to.
- **Suggested fixes.** RUBRIC Recos also has a memory section, where Claude suggests improvements and I pick the ones I like.

- A free starting point for calibrate: github.com/robonuggets/calibrate

LEVEL 1 - LONG-TERM MEMORY

AS SHOWN IN THE VIDEO

make my workspace easy for you to search, using plain text.
- keep memory.md short: one line per memory, each linked to its file
- turn claude.md into a map of my work, pointing to each department's file
- give each department a router file, like content.md, that points to its folders
- give every image a text description: save the prompt that made it, and caption old ones with a cheap model

no vector database. show me the new map before you move anything.

LEVEL 2 - SHORT-TERM MEMORY

AS SHOWN IN THE VIDEO

help me spend less of my usage on long chats.
- add to claude.md what to always keep when a chat compacts, like decisions and templates
- check my last 30 days of chats: how often did a cold cache make me pay to reload one?
- work out what keeping idle chats warm for 4 hours would have saved at api prices
- if it pays, send each idle chat a tiny ping about every 50 minutes, for up to 4 hours

show me the numbers before you set anything up.

A prompt cache belongs to one model, so the ping has to come from the same chat on the same model. Keeping the ping tiny is what keeps it cheap.

LEVEL 3 - SELF-EVOLVING MEMORY

PASTE INTO CLAUDE CODE

when i correct you twice on the same thing, your memory missed it. let's fix that.
- make a /note skill: it parks my feedback for a skill in that skill's own file
- apply parked notes only when i say apply, on the smallest model that can do it
- make a /calibrate skill: it saves what i taught you, links it to related memories, and fixes the skill it belongs to
- when you notice a lesson worth keeping, offer /calibrate instead of saving it quietly

change nothing until i approve.

Apps

Apps are the outside tools Claude can reach from the chat, plus any apps you build yourself.

L1 Connect anything

You can connect Claude to apps that have no official connector, like Adobe Premiere Pro or MyFitnessPal. Instead of digging through settings, let Claude search for the way in, in this order:

1. **The maker's own tools**, if they exist.
2. **A community CLI** (command line interface tool). Claude types one short command and gets back just the answer, so it's the lightest on your usage.
3. **The app's API** (application programming interface), usually through an API key. A step heavier, because answers come back long, with detail Claude doesn't need but still reads.
4. **A community MCP** (a plugin that lets Claude talk to an app). The heaviest, because Claude loads every MCP's full tool list at the start of each chat, even if you never use it. Keep your MCPs few.

- My /search-connectors skill does this every time. It's in Jay's Resources in the RoboNuggets classroom.

L2 Mods, for Claude Code and your browser

A mod is a small add-on, like a Chrome extension for Claude Code, that changes how it looks and works. My usage bar shows my weekly and 5-hour limits and my prompt cache, and it costs no tokens. My Claude Mods video covers more.

Take the idea beyond Claude Code. Your browser already has mods: Chrome extensions. Claude can build them for the sites you use every day. One of mine grabs the code behind any part of a website I like, so Claude can rebuild that exact look instead of guessing from a screenshot. Another saves YouTube titles and thumbnails into my database skills with a right-click, so I never leave YouTube. Extensions run as code, so they cost no tokens. To load your own, open Chrome extensions, turn on developer mode and click "Load unpacked".

L3 A micro app for every big job

Once a job is routine, ask whether a chat window is still the best way to do it. A lot of my work has moved out of the chat into small apps Claude built, running on my computer: RUBRIC Editor to review every cut, RUBRIC Carousels to approve Instagram carousels, and Generations for every image and video Claude makes.

Start with a reusable UI parts library, so Claude doesn't rebuild every piece from scratch and every app your team makes looks the same. Katie Dill, Head of Design at Stripe, says this is why design systems are having a moment again: Stripe's people and agents now build from the same system.

LEVEL 1 - CONNECT ANYTHING PASTE INTO CLAUDE CODE

connect claude to [app], even if it has no official connector.
- check in this order: the maker's own tools, a community cli, the app's api, then a community mcp
- pick the lightest one that works: a cli costs the least usage, an mcp the most
- if it needs a key, have me paste it into a file, never into this chat
- test it with one real read before you call it done

tell me what you picked and why.

LEVEL 2 - A CHROME EXTENSION PASTE INTO CLAUDE CODE

build me a small chrome extension for one job i do in my browser every week.
- ask me which job first, like saving articles i like or tracking a price
- make it run as plain code, so it costs no tokens
- save what it grabs to a folder you can read later
- show me how to load it: chrome extensions, developer mode, load unpacked

keep it to one job. we can add more later.

LEVEL 3 - A MICRO APP PASTE INTO CLAUDE CODE

find the job where i review your work in chat the most, and give it its own app.
- look through my past sessions for the review i repeat every week
- start a small ui parts library first, like a review card and a comment box, so every app reuses them
- build the smallest local app that does that one job, with a button that copies my comments back to you
- save my work to a file, so nothing is lost on restart

walk me through one real use before you call it done.

Everything I mentioned

IN THE ROBONUGGETS CLASSROOM

RUBRIC Agentic OS	The homepage from the video: micro apps, skills deck, artifacts ring, Second Brain map, and the skills and routines views. Get the RUBRIC System course.
RUBRIC Carousels	Review and approve every Instagram carousel Claude makes.
RUBRIC Recos	Claude's weekly suggestions for routines, memory and more, with a cart you copy back to Claude.
More	RUBRIC Editor, RUBRIC Generations, and skills like /search-connectors and /clean-up in Jay's Resources.
Join	skool.com/robonuggets

FREE

Ribbon (usage mod)	github.com/robonuggets/ribbon
calibrate (skill)	github.com/robonuggets/calibrate

VIDEOS

9 Claude Mods	youtube.com/watch?v=lDrAZ1wAyVs
Images and video with Claude	youtube.com/watch?v=9C4TRbucmhQ
Jev for Claude Code	youtube.com/watch?v=tTnUcSj-QPA
Agentic OS, the first ARMS levels	youtube.com/watch?v=8NSyI-npJCU
Katie Dill, Raise the Ceiling	youtu.be/GLvFTMtw4Jk

Where this goes next

You don't need every feature in this guide. The easy part is pasting the prompts: a level an evening. The skill that pays is running the loop week after week: noticing which job deserves a skill, which routine deserves a cap, which correction deserves a memory and which review deserves its own app.

If you want my exact setup, members of the RoboNuggets community get the RUBRIC Agentic OS template and the micro apps from this video. They also learn to build and sell AI systems like it for real businesses, mastering Claude Code and Claude Design along the way.

skool.com/robonuggets

I make sessions like this to help you use AI better for your business. If that's your thing, subscribe on YouTube so you catch the next one.

The ARMS checklist

Twelve levels, one test each. Tick them off as you go.

S — Skills

- [ ] **L1** What you reference every week lives in a database skill, with wildcards on.
- [ ] **L2** Fixed-step skills run on the smallest model that does the job.
- [ ] **L3** Your most-used skill gets better from your feedback.

R — Routines

- [ ] **L1** Most of your routines only ping you when something is worth flagging.
- [ ] **L2** You know what each routine costs, and the priciest one has a cap.
- [ ] **L3** A weekly routine suggests new routines for you to pick.

M — Memory

- [ ] **L1** Claude finds anything through plain-text maps, with no vector database.
- [ ] **L2** Your chats compact with a purpose, and you know when your cache is cold.
- [ ] **L3** You never correct Claude on the same thing twice.

A — Apps

- [ ] **L1** Every app you use has a way in, lightest first.
- [ ] **L2** One mod or Chrome extension saves you clicks every day.
- [ ] **L3** One weekly job lives in its own micro app.

Cheaper, faster, better. Every week.